Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**

ОТЧЕТ

по лабораторной работе № 9

Название: Web-фреймворк Django. Шаблонизатор DTL и обработка форм
Дисциплина: Языки интернет программирования

Студент ИУ6-64Б Д. О. Романов
(Группа) (Подпись, дата) (И.О. Фамилия)

Преподаватель В. И. Кузнецов
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Цель работы – ознакомиться с встроенным шаблонизатором Django Template Language (DTL), научиться передавать данные из представлений в шаблоны, создавать HTML-формы, обрабатывать POST-запросы и защищать формы от CSRF-атак.

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Django Template Language (DTL) – это встроенный язык шаблонов фреймворка Django, предназначенный для отделения логики представления от бизнес-логики приложения. Основной синтаксис DTL включает переменные, обозначаемые двойными фигурными скобками {{ variable }}, значения которых подставляются из контекста, передаваемого представлением. Теги, заключённые в скобки {% tag %}, реализуют управляющие конструкции и служебные операции: цикл {% for %} для итерации по коллекциям, условный оператор {% if %}, тег {% url %} для генерации URL по имени маршрута, тег {% csrf_token %} для вставки защитного токена в формы и тег {% static %} для подключения статических файлов. Также в DTL поддерживаются фильтры для преобразования значений переменных и механизм наследования шаблонов через {% extends %} и {% block %}, что позволяет создавать переиспользуемые макеты страниц.

В Django обработка HTTP-запросов осуществляется через объект request, передаваемый в каждое представление. Для запросов типа GET данные, отправленные в строке запроса, доступны через словарь request.GET, представляющий собой объект QueryDict. Для POST-запросов, в которых данные передаются в теле запроса, используется объект request.POST. Если

одному ключу соответствует несколько значений, например в случае группы флажков с одинаковым атрибутом name, необходимо применять метод getlist(), возвращающий список всех выбранных значений. Тип текущего запроса определяется проверкой атрибута request.method, который содержит строку “GET”, “POST” или другой HTTP-метод.

Механизм защиты от межсайтовой подделки запросов (CSRF) в Django реализован на уровне промежуточного слоя CsrfViewMiddleware, включённого по умолчанию в список MIDDLEWARE. При каждом запросе формы Middleware генерирует уникальный токен, привязанный к текущей пользовательской сессии. Для включения защиты в шаблоне формы необходимо вставить тег {% csrf_token %}, который добавляет скрытое поле с токеном. При получении POST-запроса Django автоматически сверяет значение токена в форме с токеном сессии. В случае несовпадения или отсутствия токена запрос отклоняется с кодом ошибки 403 Forbidden, что надёжно предотвращает выполнение несанкционированных действий от лица авторизованного пользователя.

3. ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1. Задание 1. Вывод ФИО и группы с использованием шаблонизатора DTL

Создано представление home, которое рендерит шаблон fio.html и передаёт в контекст ФИО студента и номер группы. В шаблоне использованы переменные DTL {{ full_name }} и {{ group }}. Настроен маршрут корневого пути ”.

Листинг 1 – представление home (views.py)

```
    # views.py
from django.shortcuts import render


def home(request):
      context = {'full_name': 'Барсиков Окак Котович',
'group': 'АК2-61'}
    return render(request, 'schedule/fio.html', context)
```

Листинг 2 – шаблон fio.html

```
    <!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>ФИО и группа</title>
</head>
<body>
    <h1>{{ full_name }}</h1>
    <p>Группа: {{ group }}</p>
          <a href="{% url 'schedule' %}">Перейти к
расписанию</a>
</body>
</html>
```

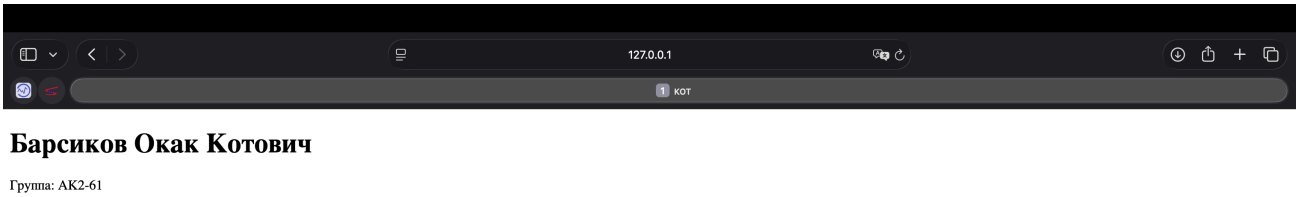

Рисунок 1 – Главная страница с ФИО и группой

3.2. Задание 2. Форма смены расписания и обработка POST-запроса

Создана страница с формой для изменения расписания. Форма содержит выпадающий список дней недели, флажки для выбора номеров пар, текстовое поле для названия предмета и выпадающий список типа занятия. Данные отправляются методом POST на обработчик update_schedule, который извлекает значения полей и выводит их в консоль сервера.

Листинг 3 – шаблон index.html (форма)

```
    <!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Смена расписания</title>
</head>
<body>
    <h1>Смена расписания</h1>
     <p>Студент: {{ full_name }} | Группа: {{ group }}</
p>
          <form  action="{%  url  'update_schedule'  %}"
method="post">
        {% csrf_token %}
        <label for="day">День недели:</label>
        <select name="day" id="day">
                <option value="Понедельник">Понедельник</
option>
            <option value="Вторник">Вторник</option>
            <option value="Среда">Среда</option>
            <option value="Четверг">Четверг</option>
            <option value="Пятница">Пятница</option>
            <option value="Суббота">Суббота</option>
        </select><br><br>
```

```
        <label>Номера пар:</label><br>
           <input type="checkbox" name="para" value="1">
1<br>
           <input type="checkbox" name="para" value="2">
2<br>
           <input type="checkbox" name="para" value="3">
3<br>
           <input type="checkbox" name="para" value="4">
4<br>
           <input type="checkbox" name="para" value="5">
5<br><br>

        <label for="subject">Предмет:</label>
                     <input  type="text"  name="subject"
id="subject"><br><br>

        <label for="lesson_type">Тип занятия:</label>
        <select name="lesson_type" id="lesson_type">
            <option value="Лекция">Лекция</option>
            <option value="Практика">Практика</option>
              <option value="Лабораторная">Лабораторная</
option>
            <option value="Семинар">Семинар</option>
        </select><br><br>

        <button type="submit">Отправить</button>
    </form>
    <br>
    <a href="{% url 'home' %}">На главную</a>
</body>
</html>
```

Листинг 4 – представление update_schedule (views.py)

```
    # views.py
def update_schedule(request):
    if request.method == 'POST':
        day = request.POST.get('day')
        paras = request.POST.getlist('para')
        subject = request.POST.get('subject')
        lesson_type = request.POST.get('lesson_type')

        print(f'day {day}')
        print(f'paras {paras}')
        print(f'subject {subject}')
        print(f'lesson_type {lesson_type}')
        return redirect('index')

    return redirect('schedule')
```

Листинг 5 – файл urls.py приложения

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('schedule/', views.index, name='schedule'),
         path('update-schedule/',  views.update_schedule,
name='update_schedule'),]
```

Листинг 6 – головной urls.py проекта

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('schedule.urls')),]
```

Листинг 7 – фрагмент settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    'schedule',]

TEMPLATES = [{
                                        "BACKEND":
"django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
                                "context_processors":
["django.template.context_processors.request",

"django.contrib.auth.context_processors.auth",

"django.contrib.messages.context_processors.messages",],
},},]
```

Листинг 8 – представление index (views.py)

```
def index(request):
        context = {'full_name': 'Барсиков Окак Котович',
'group': 'АК2-61'}
          return render(request, 'schedule/index.html',
context)
```

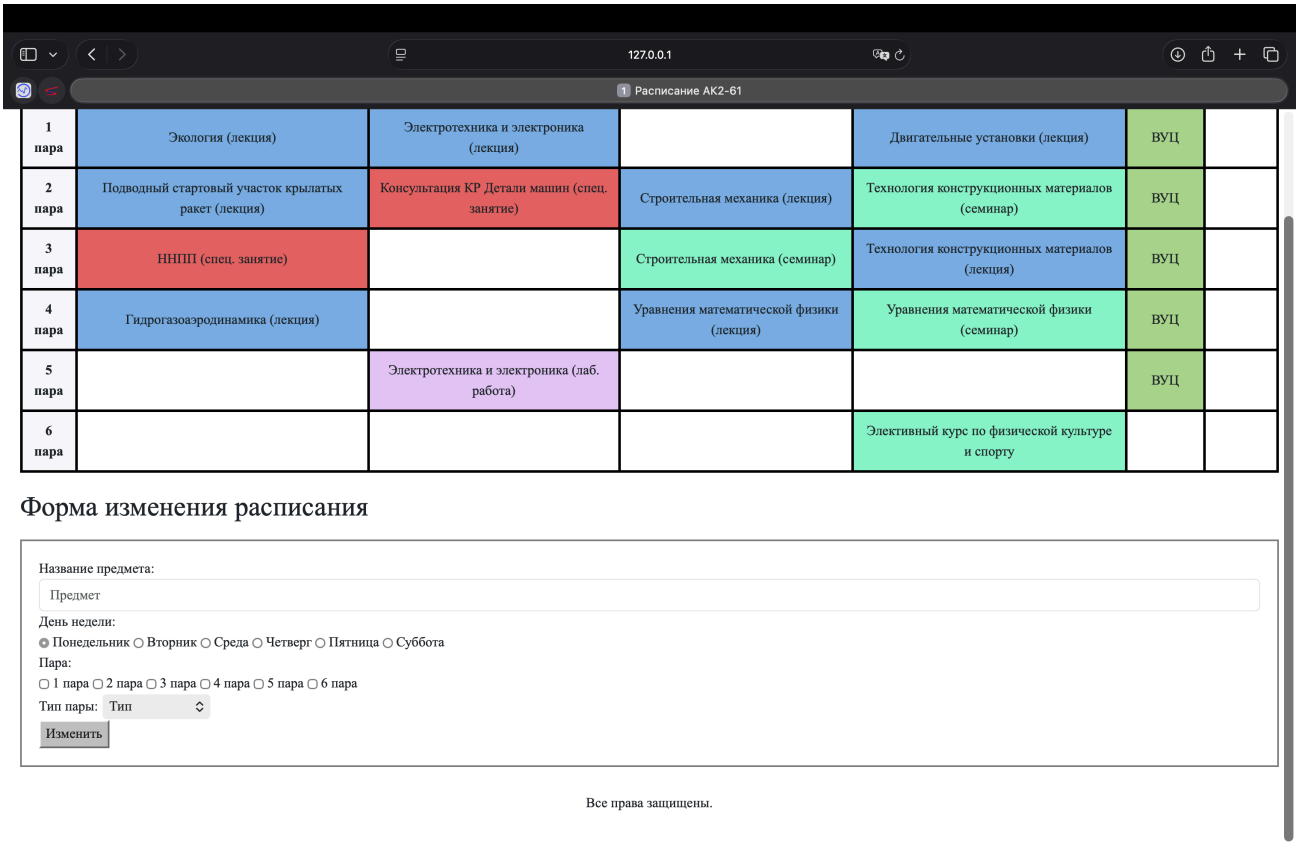

Рисунок 2 – Страница с формой смены расписания

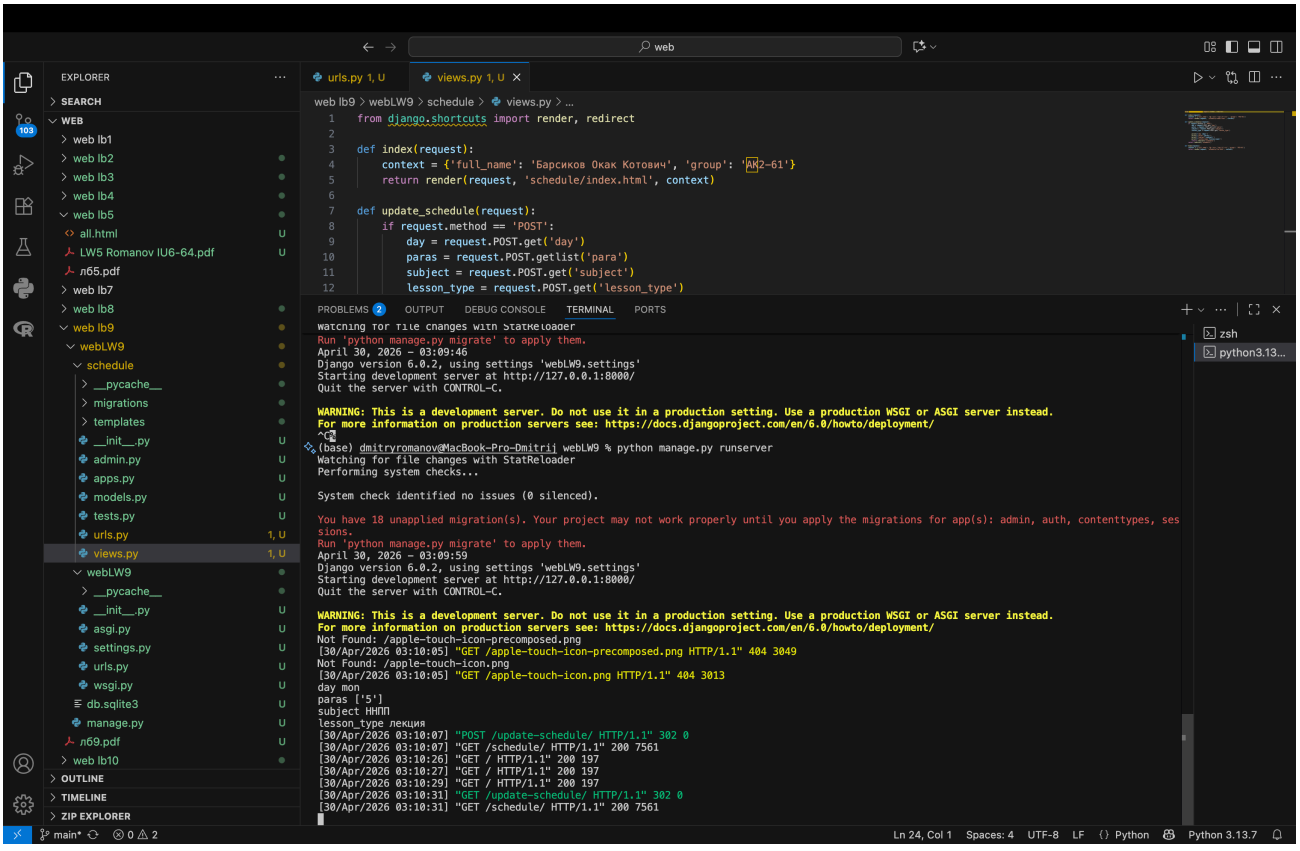

Рисунок 3 – Консоль сервера с выводом данных формы

1. ВЫВОДЫ

В ходе лабораторной работы были изучены и на практике применены возможности шаблонизатора Django Template Language. Освоены передача контекста из представления в шаблон, синтаксис переменных {{ }} и тегов {% %}. Созданы HTML-формы с различными типами полей, реализована обработка POST-запросов, извлечение данных через request.POST и вывод в консоль сервера. Применён механизм CSRF-защиты с помощью тега {% csrf_token %}. Настроена двухуровневая маршрутизация URL.

5. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1: Что такое шаблонизатор DTL и как он работает?

Ответ: Django Template Language (DTL) – встроенный язык шаблонов Django. Переменные {{ var }} подставляются из контекста. Теги {% tag %} реализуют логику (for, if, url, csrf_token). Шаблон компилируется, затем рендерится с подстановкой значений.

Вопрос 2: Как в Django обрабатываются GET и POST запросы?

Ответ: Через объект request. GET-параметры – request.GET (QueryDict), POST-данные – request.POST. Для множественных значений используется getlist(). Тип запроса проверяется через request.method.

Вопрос 3: Как защитить форму от CSRF?

Ответ: Включить CsrfViewMiddleware в MIDDLEWARE и добавить {% csrf_token %} внутрь формы. Токен привязан к сессии, проверяется автоматически. Без токена запрос отклоняется с 403.